



Compte rendu de TP 6

SPRING MVC



Module : *Ingestion et Stockage des données.*

Filière : *Transformation Digitale & Intelligence Artificielle 2.*

Année : *2025/2026.*

Réalisé par :

- Es-saouiqui Amine.

Supervisée par :

- Dr. Cherradi Mohamed



Table de matière :

I.	Introduction :	4
II.	Architecture du Spring MVC :	4
III.	Architecture technique :	5
IV.	Réalisation technique :	6
	1)- Organisation du projet :	6
	a) Couche Backend (src/main/java) :	7
	b) Couche Configuration (src/main/resources) + (src/main/webapp/WEB-INF/) :	8
	c) Couche Frontend (src/main/webapp) :	8
	2)- Configuration du fichier pom.xml :	8
	3)- Configuration du fichier hibernate.cfg.xml :	10
	3)- Définition des entités :	11
	a) – User.java :	11
	b)- Produit.java :	13
	4)- Couche DAO :	14
	5)- Couche Service :	14
	6)- Couche controllers :	14
	a)- UserController.java :	15
	b)- ProduitController.java :	17
	7)- Injection de Dépendances et Résolution de Vues :	20
	a) - Configuration des couches DAO et Service (XML) :	20
	b)- Injection des services dans la couche Contrôleur :	21
	c) - Résolution des Vues (View Resolver) :	21
	8)- Les filtres :	22
	9)- Les vues :	22
V.	Conclusion :	22



Table de Figures :

Figure 1 : Architecture du Spring MVC	5
---	---

Table de Scripts :

Script 1 : Configuration du fichier pom.xml	10
Script 2 : Configuration du fichier hibernate.cfg.xml	11
Script 3 : class User.java	12
Script 4 : Produit.java	14
Script 5 : classe UserController.java	16
Script 6 : classe ProduitController.java	18
Script 7 : Fichier spring-beans.xml	20
Script 8 : configuration du fichier application-servlet-config.xml	22



I. Introduction :

Ce projet porte sur le développement d'une application web de gestion de stock supportant l'ensemble des opérations **CRUD** sur les produits, initialement réalisée en **Java EE** pur. L'architecture a été migrée vers **Spring MVC**, tout en conservant une approche explicite de la persistance via **Hibernate** standard. Cette migration conserve le pattern **Front Controller**, mais le délègue au **DispatcherServlet** natif de Spring, et remplace la gestion manuelle des sessions, des filtres et du routage par les mécanismes d'injection de dépendances, de transactions déclaratives et de sécurité intégrée de l'écosystème Spring. Cette approche modernise le code tout en garantissant une séparation rigoureuse des responsabilités et une meilleure maintenabilité.

II. Architecture du Spring MVC :

Le framework **Spring MVC** implémente nativement le design pattern Front Controller, centralisant le traitement des requêtes HTTP via une servlet pré-configurée et hautement optimisée : le **DispatcherServlet**. Contrairement à l'implémentation manuelle du TP précédent (servlet maison, pattern Command codé avec réflexion Java, Dispatcher personnalisé), **Spring MVC** propose une architecture standardisée simple à utiliser via les annotations.

Ses rôles principaux :

- **Centralisation** : Toutes les requêtes HTTP sont interceptées par le **DispatcherServlet**, qui agit comme point d'entrée unique de l'application.
- **Routage intelligent (HandlerMapping)** : Associe automatiquement une URL à la méthode d'un **@ Controller** grâce aux annotations **@ RequestMapping**, **@ GetMapping** ou **@ PostMapping**.
- **Exécution & Binding (HandlerAdapter)** : Injecte les paramètres HTTP dans les arguments des méthodes, gère les conversions de types (**@ RequestParam**, **@ PathVariable**, **@ ModelAttribute**) et applique la validation déclarative.
- **Résolution de vue (ViewResolver)** : Transforme un nom logique retourné par le contrôleur (ex: "home") en chemin physique vers une JSP (**/WEB-INF/views/home.jsp**).

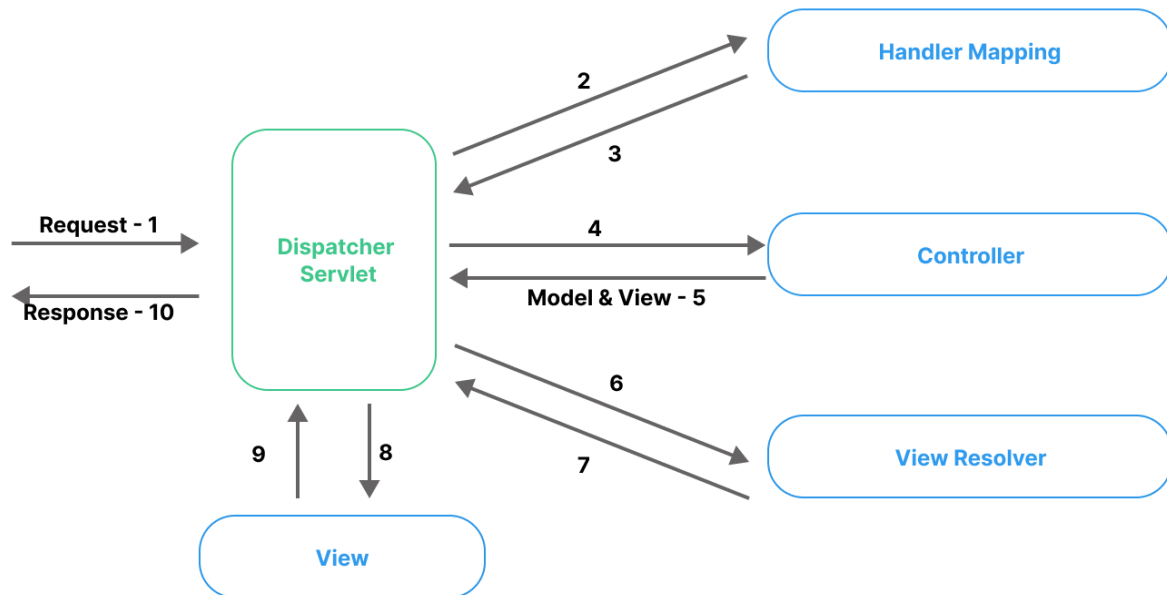


Figure 1 : Architecture du Spring MVC

III. Architecture technique :

L'application conserve les mêmes fonctionnalités métier, mais l'architecture technique évolue vers les standards de l'écosystème **Spring MVC**.



IV. Réalisation technique :

1)- Organisation du projet :

Le projet respecte l'arborescence standard d'une application web de type **Maven**, permettant une séparation nette entre le code source, les ressources et l'interface utilisateur.





```
└─WEB-INF
    application-servlet-config.xml
    spring-beans.xml
    web.xml
```

a) Couche Backend (src/main/java)

Ce répertoire constitue le cœur de la logique applicative, organisé en plusieurs packages respectant une architecture en couches :

- **entities** : Regroupe les objets de données . Les classes comme **Produit.java** et **User.java** définissent la structure des données et facilitent leur circulation entre les différentes couches de l'application.
- **dao** : Ce package isole la logique de persistance. Il repose sur des interfaces (**ProduitDAO**, **UserDAO**) et leurs implémentations (**ProduitDAOImpl**, **UserDAOImpl**), en s'appuyant sur **Hibernate** pour assurer le mapping objet-relationnel et l'accès à la base de données.
- **services** : Joue le rôle d'intermédiaire entre les contrôleurs et les DAO. Il contient la logique métier, notamment les règles de gestion et les validations nécessaires avant toute interaction avec la base de données.
- **controllers** : Regroupe les classes responsables du traitement des requêtes métier. Elles définissent les actions à exécuter en fonction des commandes reçues et participent à l'organisation du flux applicatif.
- **filters** : Package dédié à la gestion des aspects transversaux. **AuthFilter** vérifie l'authentification des utilisateurs, **RoleFilter** contrôle les autorisations, et **HibernateSessionFilter** gère le cycle de vie des sessions Hibernate.
- **utils** : Contient des classes utilitaires, notamment **HibernateUtils** qui permet de configurer et de gérer la **SessionFactory** pour l'accès à la base de données.



b) Couche Configuration (src/main/resources) + (src/main/webapp/WEB-INF)

Ces répertoire regroupe les fichiers de configuration nécessaires au bon fonctionnement de l'application :

- **src/main/resources/hibernate.cfg.xml** : Fichier principal de configuration de Hibernate, définissant les paramètres de connexion à la base de données ainsi que les propriétés ORM.
- **src/main/webapp/webapp/WEB-INF/spring-beans.xml** : Contexte racine chargé par **ContextLoaderListener**. Il décrit les beans et le mécanisme d'injection.
- **src/main/webapp/webapp/WEB-INF/application-servlet-config.xml** : Contexte servlet chargé par le **DispatcherServlet**. permet de scanner les beans annotés et configure le **ViewResolver** pour les JSP.

c) Couche Frontend (src/main/webapp)

Ce répertoire représente la partie visible de l'application, gérée par le serveur web :

- **views** : Contient les pages JSP (comme **home.jsp** et **error.jsp**), utilisées pour afficher les données dynamiques à l'utilisateur via JSTL.
- **index.jsp** : Point d'entrée principal de l'application, servant de page d'accueil ou de redirection initiale.

2)- Configuration du fichier pom.xml :

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/maven-v4_0_0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>tp6</artifactId>
  <packaging>war</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>tp6</name>
  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
    <spring.version>5.3.34</spring.version>
  </properties>
  <dependencies>
```



```
<!-- TEST -->
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.13.2</version>
  <scope>test</scope>
</dependency>
<!-- HIBERNATE -->
<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>
<!-- MYSQL -->
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <version>8.0.33</version>
</dependency>
<!-- JSTL -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>jstl</artifactId>
  <version>1.2</version>
</dependency>
<!-- SERVLET API (TOMCAT 9 = javax) -->
<dependency>
  <groupId>javax.servlet</groupId>
  <artifactId>javax.servlet-api</artifactId>
  <version>4.0.1</version>
  <scope>provided</scope>
</dependency>
<!-- SPRING (IMPORTANT: 5.x not 7.x) -->
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-core</artifactId>
  <version>${spring.version}</version>
</dependency>
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-beans</artifactId>
  <version>${spring.version}</version>
</dependency>
```



```

    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-context</artifactId>
      <version>${spring.version}</version>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-web</artifactId>
      <version>${spring.version}</version>
    </dependency>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-webmvc</artifactId>
      <version>${spring.version}</version>
    </dependency>
  </dependencies>
  <build>
    <finalName>tp6</finalName>
  </build>
</project>

```

Script 1 : Configuration du fichier pom.xml

3)- Configuration du fichier hibernate.cfg.xml :

Pour la configuration de la persistance, nous avons conservé exactement les mêmes paramètres de connexion et de comportement ORM que dans le TP précédent. La différence majeure réside dans la suppression des fichiers de mapping XML (Produit.hbm.xml, User.hbm.xml) au profit d'**annotations standards** directement placées dans les classes entités.

Cette approche, supportée nativement par Spring, permet de centraliser la configuration de Hibernate tout en externalisant le mapping directement dans le code métier.

```

<?xml version='1.0' encoding='utf-8'?>
<hibernate-configuration>
  <!-- a SessionFactory instance listed as /jndi/name -->
  <session-factory>
    <!-- properties -->
    <!-- JDBC Database Connection Settings -->
    <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/tp4_db</property>
    <property name="hibernate.connection.username">amine</property>
    <property name="hibernate.connection.password">1234</property>

```



```
<property
name="dialect">org.hibernate.dialect.MySQL8Dialect</property>
<property name="show_sql">false</property>
<!-- Enable Hibernate to automatically create/drop tables -->
<property name="hibernate.hbm2ddl.auto">update</property>
<!-- Additionnal : Relating a session context to one thread -->
<property
name="hibernate.current_session_context_class">thread</property>
<mapping class="entities.User" />
<mapping class="entities.Produit" />
</session-factory>
</hibernate-configuration>
```

Script 2 : Configuration du fichier hibernate.cfg.xml

3)- Définition des entités :

Pour la définition des entités, nous avons conservés les mêmes classes de TP précédent.

a) – User.java:

```
package entities;

import javax.persistence.Entity;
import javax.persistence.EnumType;
import javax.persistence.Enumerated;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;
@Entity
@Table(name="USERS")
public class User {

    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;
    private String username;
    private String password;

    @Enumerated(EnumType.ORDINAL)
    private Roles role;
```



```
public User(){}  
public User(String username,String password,Roles role){  
    this(username,password);  
    this.role = role;  
}  
public User(String username,String password){  
    this.username = username;  
    this.password = password;  
}  
public Long getId() {  
    return id;  
}  
public String getUsername() {  
    return username;  
}  
public void setId(Long id){  
    this.id = id;  
}  
public void setUsername(String username) {  
    this.username = username;  
}  
public String getPassword() {  
    return password;  
}  
public void setPassword(String password) {  
    this.password = password;  
}  
public Roles getRole() {  
    return role;  
}  
public void setRole(Roles role) {  
    this.role = role;  
}  
}
```

Script 3 : class User.java

**b)- Produit.java :**

```
package entities;

import javax.persistence.Entity;
import javax.persistence.GeneratedValue;
import javax.persistence.GenerationType;
import javax.persistence.Id;
import javax.persistence.Table;

@Entity
@Table(name="PRODUITS")
public class Produit {
    @Id
    @GeneratedValue(strategy=GenerationType.IDENTITY)
    private Long id;

    private String nom;
    private String description;
    private Double prix;

    public Produit(){}
    public Produit(String nom,String description,Double prix){
        this.nom = nom;
        this.description = description;
        this.prix = prix;
    }
    public void setId(Long id) {
        this.id = id;
    }
    public void setNom(String nom){
        this.nom = nom;
    }
    public void setDescription(String description){
        this.description = description;
    }
    public void setPrix(Double prix){
        this.prix = prix;
    }
    public Long getId() {
        return id;
    }
}
```



```
public String getNom() {  
    return nom;  
}  
public String getDescription() {  
    return description;  
}  
public Double getPrix() {  
    return prix;  
}  
}
```

Script 4 : Produit.java

4)- Couche DAO :

Pour la **couche DAO**, il n'y a aucun changement au niveau des classes.

5)- Couche Service:

La **couche Service** reprend les mêmes définitions et structures de classes que celles implémentées dans le TP précédent. Elle conserve donc la même organisation ainsi que les mêmes responsabilités, sans modification majeure au niveau de la logique métier.

6)- Couche controllers :

Le composant central de notre logique de navigation est la classe annotée avec **@Controller**.

L'utilisation de cette annotation nous a permis de :

- *Intercepter les requêtes HTTP entrantes envoyées par le client (navigateur).*
- *Mapper les URLs spécifiques aux méthodes correspondantes via les annotations **@GetMapping** et **@PostMapping**.*
- *Assurer le lien entre la couche service (logique métier) et la couche présentation.*



a)- UserController.java :

Pour gérer l'authentification et le cycle de vie de la session utilisateur, nous avons implémenté la classe `UserController`. L'utilisation de l'annotation `@Controller` a été ici déterminante pour structurer les flux de connexion et de déconnexion.

```
package controllers;

import javax.servlet.http.HttpSession;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;

import entities.User;
import services.UserService;

@Controller
@RequestMapping("/users")
public class UserController {

    private UserService service;

    public UserController(UserService service){
        this.service = service;
    }

    @PostMapping("/login")
    public String login(@ModelAttribute("user") User userForm, HttpSession session){
        User user = service.login(userForm.getUsername(),
userForm.getPassword());
        if(user != null){
            session.setAttribute("user", user);
            return "redirect:/views/home.jsp";
        }
        return "redirect:/index.jsp";
    }
}
```



```
@GetMapping("/logout")
public String logout(HttpSession session){
    session.invalidate();
    return "redirect:/index.jsp";
}
}
```

Script 5 : classe UserController.java

L'analyse de notre code met en évidence plusieurs fonctionnalités critiques de **Spring MVC** :

- **Mapping de niveau classe (@RequestMapping("/users"))** : Cette annotation permet de préfixer toutes les routes de la classe. Ainsi, les méthodes de **login** et **logout** sont accessibles via les URLs **/users/login** et **/users/logout**.
- **Injection de dépendances par constructeur** : Nous avons injecté le **UserService** directement via le **constructeur** (sans utiliser **@Autowired**). C'est la recommandation actuelle de Spring pour garantir l'immuabilité et faciliter les tests unitaires.
- **Gestion de la Session (HttpSession)** : Le contrôleur reçoit l'objet **session** en paramètre. Cela nous permet de stocker l'objet **User** après une authentification réussie, rendant l'utilisateur **"connecté"** sur l'ensemble de l'application.

Nous avons implémenté deux types d'interactions :

- **La Connexion (@PostMapping("/login"))** :
 - Utilisation de **@ModelAttribute** pour lier automatiquement les champs du formulaire HTML à l'objet **User**.
 - Logique conditionnelle : si le service valide l'utilisateur, on redirige vers l'accueil (**home.jsp**), sinon on renvoie l'utilisateur vers la page de connexion avec un **redirect**.
- **La Déconnexion (@GetMapping("/logout"))** :
 - Utilisation de **session.invalidate()** pour détruire les données de session et assurer la sécurité de la sortie.



Voici un tableau récapitulatif :

Annotation	URL	Méthode HTTP	Rôle
@PostMapping	<i>/users/login</i>	<i>POST</i>	<i>Traitement sécurisé du formulaire de connexion</i>
@GetMapping	<i>/users/logout</i>	<i>GET</i>	<i>Destruction de la session et déconnexion</i>

b)- ProduitController.java :

La classe **ProduitController** gère l'ensemble du cycle de vie des produits. Contrairement au **UserController** qui gère des états de session, ce contrôleur se concentre sur l'échange de données entre le service et les templates.

```
package controllers;

import java.util.List;

import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;

import entities.Produit;
import services.ProduitService;

@Controller
@RequestMapping("/produits")
public class ProduitController {

    private ProduitService service;

    public ProduitController(ProduitService service){
        this.service = service;
    }
}
```



```
@GetMapping("/list")
public String listProduits(Model model){
    List<Produit> produits = service.getAllProduits();
    model.addAttribute("Produits",produits);
    return "home";
}

@GetMapping("/get")
public String getProduit(@RequestParam(name="id",defaultValue="0") Long
id,Model model){
    Produit p = service.getProduitById(id);
    if (p!=null){
        model.addAttribute("Produits", List.of(p));
        return "home";
    }else return "redirect:/produits/list";
}

@PostMapping("/add")
public String createProduit(@ModelAttribute("produit") Produit produit){
    this.service.addProduct(produit);
    return "redirect:/produits/list";
}

@PostMapping("/update")
public String updateProduct(@ModelAttribute ("produit") Produit produit){
    this.service.updateProduit(produit);
    return "redirect:/produits/list";
}

@GetMapping("/delete")
public String deletePorduct(@RequestParam("id") Long id){
    this.service.deleteProduit(id);
    return "redirect:/produits/list";
}

@GetMapping("/edit")
public String preparEdit(@RequestParam("id") Long id,Model model){
    Produit p = this.service.getProduitById(id);
    model.addAttribute("produitEdit",p);
    return "forward:/produits/list";
}
}
```

Script 6 : classe ProduitController.java



Pour transmettre les données à afficher, nous avons utilisé l'interface **Model**.

- Dans la méthode **ListProduits**, l'instruction **model.addAttribute("Produits", produits)** permet d'injecter la liste des objets récupérés depuis la base de données vers la vue **home.jsp**.

Nous avons utilisé deux approches pour récupérer les informations envoyées par l'utilisateur :

- **@RequestParam** : Utilisé pour les actions de suppression (**/delete**) et d'édition (**/get**, **/edit**). Cela permet d'extraire l'identifiant du produit directement depuis l'URL.
- **@ModelAttribute** : Utilisé dans les méthodes **createProduit** et **updateProduct**. Cette annotation simplifie grandement le code en liant automatiquement tous les champs d'un formulaire HTML à un objet Java **Produit**.

Un aspect technique intéressant de ce TP réside dans la gestion des flux de navigation :

- **redirect:/produits/list** : Utilisé après une modification (ajout, suppression, mise à jour). Cela force une nouvelle requête GET pour rafraîchir la liste et évite de répéter l'action en cas d'actualisation de la page.
- **forward:/produits/list** : Utilisé dans la méthode **preparEdit**. Le forward permet de transférer la requête en interne au serveur sans changer l'URL du navigateur. Cela est ici indispensable pour conserver l'objet **produitEdit** dans le modèle et l'afficher dans le formulaire de modification sur la même page que la liste.

Voici un tableau récapitulatif :

Action	Méthode HTTP	URL	Annotation clé
Affichage	GET	/produits/list	Model
Recherche	GET	/produits/get	@RequestParam
Ajout	POST	/produits/add	@ModelAttribute
Mise à jour	POST	/produits/update	@ModelAttribute
Suppression	GET	/produits/delete	@RequestParam



7)- Injection de Dépendances et Résolution de Vues :

Dans ce projet, nous avons mis en place une gestion *hybride* des dépendances, combinant la **configuration XML** et les **annotations**. Ce choix permet de séparer clairement la déclaration des composants d'**infrastructure (DAO et Services)** de la logique de **présentation (Contrôleurs)**.

a) - Configuration des couches DAO et Service (XML) :

Pour la persistance et la logique métier, nous avons utilisé le fichier **spring-beans.xml**. Ce fichier sert de registre central pour le *conteneur IoC*.

```
<?xml version="1.0" encoding="UTF-8"?>
<beans>
<!-- Couche DAO -->
<bean id="ProduitDAO" class="dao.ProduitDAOImpl"/>
<bean id="UserDAO" class="dao.UserDAOImpl"/>
<!-- Couche Service -->
  <!-- Injection par constructeur -->
  <bean id="ProduitService" class="services.ProduitServiceImpl">
    <constructor-arg index="0" ref="ProduitDAO" />
  </bean>
  <bean id="UserServices" class="services.UserServiceImpl">
    <constructor-arg index="0" ref="UserDAO" />
  </bean>
</beans>
```

Script 7 : Fichier spring-beans.xml

- **Déclaration des Beans** : Chaque implémentation (ex: *ProduitDAOImpl*) est déclarée avec une balise **<bean>**.
- **Injection par Constructeur (XML)** : Nous avons utilisé la balise **<constructor-arg>** pour injecter les **DAO** dans les **services**.



b)- Injection des services dans la couche Contrôleur :

Alors que les couches de données et de services sont centralisées dans `spring-beans.xml`, la couche de présentation est pilotée par le fichier `application-servlet-config.xml`.

C'est ce fichier qui fait le lien entre les requêtes HTTP et nos classes Java.

La liaison du couche service avec la couche controllers s'appuie sur le mécanisme de **Component Scanning** déclaré dans le fichier `application-servlet-config.xml` via la balise `<context:component-scan base-package="controllers" />`.

Ce dispositif permet à Spring de détecter automatiquement les classes annotées avec **@ Component ou bien ses stéréotypes**, telles que `UserController` et `ProduitController`, pour les instancier en tant que `beans`.



Un aspect crucial de cette architecture réside dans **l'injection de dépendances inter-contextes** :

*Bien que les **services** soient définis **séparément** dans le fichier `spring-beans.xml`, le **conteneur Spring** parvient à les résoudre et à les injecter **implicitement** dans les constructeurs des contrôleurs. Ainsi, lors de l'instanciation du `ProduitController`, la dépendance `ProduitService` est automatiquement récupérée depuis le contexte parent pour finaliser l'assemblage des composants.*

c) - Résolution des Vues (View Resolver) :

Pour transformer les noms logiques retournés par nos méthodes (*comme "home"*) en ressources physiques, nous avons configuré l'`InternalResourceViewResolver` à l'intérieur de ce même fichier `application-servlet-config.xml`.

Ce composant est essentiel pour transformer le retour `"home"` du `ProduitController` en un chemin réel : `/WEB-INF/views/home.jsp`.



```
<?xml version="1.0" encoding="UTF-8"?>
<beans >
<!-- Scanning du package Controller -->
<context:component-scan base-package="controllers"/>
<!-- Resolution de la vue -->
<bean class="org.springframework.web.servlet.view.InternalResourceViewResolver">
<property name="prefix" value="/views/" />
<property name="suffix" value=".jsp" />
</bean>
</beans>
```

Script 8 : configuration du fichier application-servlet-config.xml

8)- Les filtres :

Dans un souci de continuité et de réutilisation logicielle, nous avons intégré à ce projet les **Filtres** implémentés lors du **TP précédent**.

9)- Les vues :

Pour la partie interface utilisateur, nous avons fait le choix de réutiliser les **fichiers JSP** avec les balises **JSTL** conçus lors du **TP précédent**. Cette approche démontre la flexibilité de **Spring MVC**, qui s'intègre parfaitement avec des technologies de présentation existantes.

V. Conclusion :

Ce projet a permis de mettre en pratique une architecture web structurée autour du framework **Spring MVC**, en consolidant le découpage en couches (Controller, Service, DAO et Vue).

L'adoption de l'annotation **@ Controller** et du mécanisme de **Front Controller** via la **DispatcherServlet** a permis de simplifier la gestion des flux de navigation et le mapping des requêtes.

L'intégration hybride entre la **configuration XML** (pour la couche métier et DAO) et **les annotations** (pour la couche présentation) a démontré la flexibilité de Spring pour assembler des composants découplés.

L'utilisation du **ViewResolver** et de l'**injection de dépendances** par constructeur a renforcé la robustesse du système, tout en permettant la réutilisation efficace des **filtres** de sécurité et des vues JSP issus des développements précédents.



Ainsi, ce travail a permis de mieux comprendre les mécanismes avancés de l'écosystème **Spring** et l'importance de l'injection de dépendances pour garantir une application évolutive, maintenable et conforme aux standards industriels actuels du développement **Java EE**.